

# Apunte 3

---

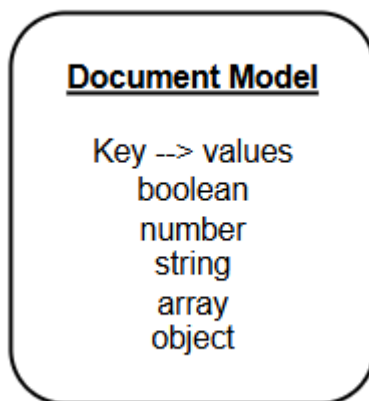
Priscilla Romero Barquero - 2023332718

---

## Mongo

Mongo actualmente utiliza el document model, es el corazón de Mongo y de todas las bases de datos documentales.

Este modelo se basa en key-value pairs, es decir, en pares de clave y valor. Cada valor representa los datos y puede mapearse a diferentes tipos como: **boolean**, **number**, **string**, **array** u **object**. El document model básicamente consiste en representar los datos como documentos JSON.



Es posible representar un JSON dentro de otro JSON, y dentro de otro, y así sucesivamente. A esto se le conoce como JSON anidado, lo que en términos prácticos corresponde a un objeto. ¿Es posible representar cualquier documento en JSON? Sí, y su nombre proviene de *JavaScript Object Notation*.

En MongoDB, una fila (row) equivale a un documento (doc). El equivalente de un documento en una base de datos documental es una fila dentro de una base de datos SQL; ambos son intercambiables.

Este modelo permite modificar la estructura del documento y añadir campos dinámicamente, lo cual significa que dentro de una base de datos documental podemos:

- Agregar o quitar campos libremente.
- Tener documentos con diferentes formas o estructuras, es decir, con distintos campos entre sí.

## Límite de tamaño en Mongo

Mongo tiene una cuota máxima de 16 MB por documento. Si un documento supera este tamaño, no puede almacenarse en MongoDB.

Normalmente, cuando tenemos volúmenes muy grandes de documentos, es porque contienen datos binarios. Los datos binarios incluyen elementos como:

- Imágenes
- Videos

- Audios
- Archivos ejecutables
- Archivos comprimidos (por ejemplo `.zip` o `.tar`)
- Cualquier tipo de archivo que no puede abrirse o leerse en texto plano

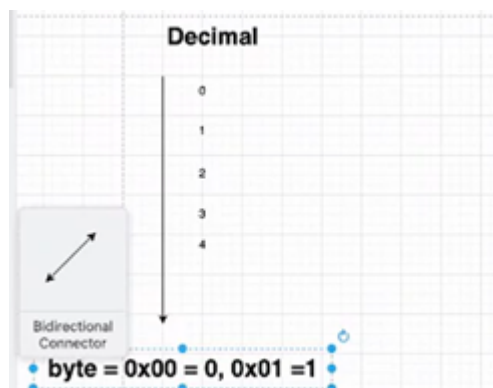
Cuando necesitamos incluir datos binarios dentro de un documento JSON, podemos convertirlos a Base64. Esto significa tomar una representación binaria y transformarla en una representación textual, codificando la información del archivo binario en los **64 caracteres de la tabla ASCII**.

Por ejemplo, si tenemos 1 MB de datos binarios, al convertirlos a Base64 el tamaño aumentará aproximadamente a 1.5 MB.

¿Por qué ocurre esto?

Pensemos que trabajamos con un lenguaje decimal, el cual utiliza símbolos como 0, 1, 2, 3, 4, y así sucesivamente. El lenguaje decimal define que tenemos 10 símbolos posibles.

Supongamos que con esos 10 símbolos representamos 1 byte, entonces 1 byte equivale a la representación de 10 símbolos.



byte = 0x00 = 0, 0x01 = 1

Ahora, ¿qué pasa si necesitamos hacer una representación pero no podemos usar toda la tabla ASCII para representar los datos?

En ese caso, el 0 y el 1 siguen midiendo un byte.

Si decimos que el sistema decimal tiene 10 caracteres, debemos preguntarnos con cuántos bits podemos representar esos 10 caracteres.

Si tengo dos símbolos, que son el 0 y el 1, puedo representar 2 combinaciones.

Si tengo dos símbolos, puedo representar 4; si tengo tres símbolos, puedo representar 9; y si tengo cuatro símbolos, puedo representar 16.

Entonces, ¿qué me dice Base64? Base64 me dice que puedo representar hasta 256 caracteres, pero solo me permite usar 64. Por eso debo calcular cuántos caracteres necesito en una representación para poder codificar con 256. Cuando decido almacenar datos en Base64, la información que estoy guardando crece, porque me acerco más al límite de los 64 caracteres permitidos por esta codificación.

## Manejo de datos binarios y búsquedas en Mongo

Un dato binario no tiene impacto en las búsquedas.

Por ejemplo, si le pedimos a Mongo que busque todos los campos que digan "Nereo", no lo va a encontrar si el valor está almacenado en formato binario. Mongo guarda los datos de texto plano como strings, y al realizar la búsqueda, si no encuentra coincidencias textuales, simplemente determina que no hay un match. Esto significa que estamos desperdiciando el potencial de la base de datos, ya que estos datos no se pueden indexar de forma útil.

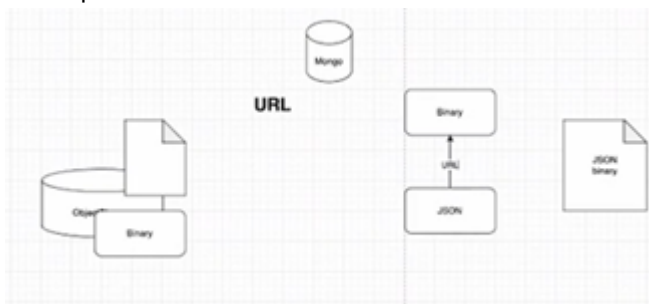
Una solución es ser inteligentes en la forma en que guardamos los datos. Por ejemplo, podemos usar un object storage como S3 o GCS.

Cada documento que se guarda en un object storage genera una URL, así que si tenemos un JSON que contiene datos binarios, podemos separar esa parte binaria del documento JSON.

El proceso sería el siguiente:

1. Tomamos el documento con el dato binario.
2. Guardamos ese dato binario en el object storage.
3. El object storage genera una URL.
4. Esa URL se almacena dentro del documento JSON.

De esta forma, la información binaria queda almacenada en un sistema diseñado específicamente para ese tipo de datos, mientras que el JSON solo contiene la referencia, la URL que indica dónde se encuentra el archivo. Así el documento se mantiene pequeño, no crece por la codificación de los binarios, y los campos del JSON pueden utilizarse en consultas e índices de manera eficiente.



## Almacenamiento de múltiples documentos JSON

Los docs JSON que vamos a guardar se pueden imaginar de la forma más simple. Si tenemos múltiples de estos, almacenarlos así es algo sumamente complejo, como el ejemplo de la imagen. Es complejo porque el simple hecho de tener esa info almacenada va a traer ciertas complicaciones, por ejemplo cómo crece el tamaño del documento.

```
{
  "nodes": [{
    "id": 45
  }],
  "edges": [
    {
      "src": 45,
      "dst": 32,
      "weight": 78
    }
  ]
}
```

Las comillas son solo 12 caracteres.

Con los puntos son 18 caracteres.

Con las llaves ya serían 26 caracteres.

Con todas las letras ya serían aprox 50 caracteres.

Todos se van a traducir como 50 bytes. Si tengo un millón de docs que tengan la misma forma, serían 50 millones de bytes, serían 0.46 gigas. Esto es desperdicio de almacenamiento.

Ponerlos en una colección causaría que hay que meterse en todos los documentos y eso sería muy ineficiente para la base de datos, entonces no se maneja así. La alternativa es agarrar un solo archivo y tener un archivo grandote en el cual voy pegando un archivo detrás del otro. De esta manera me va a quedar conectado al siguiente o va a ser una línea del archivo grandote. Esto permite abrir un solo archivo y moverse sobre él, en vez de abrir uno por uno.

¿Qué pasa en ese caso? Cada doc va a ser un JSON y se sabe que el JSON va a empezar con llaves, tiene nombres con comillas dobles, etc. Entonces tengo que construir algo que pueda moverse sobre el archivo y, conforme se mueve, identificar dónde inicia y dónde termina el doc y después moverse al siguiente. Pero como es textual, no tengo ningún tipo de info que me pueda guiar de cómo se almacenan los docs. Solo puedo ir leyendo fila por fila e ir interpretando hasta encontrar el doc que andamos buscando. Ese es el último recurso al que deberíamos acudir: hacer un scan, ir uno por uno hasta encontrar, revisar todo, y en el peor de los casos estaría de último.

Con esto, se puede decir que JSON no es amigable para hacer manejo de datos a nivel de almacenamiento. Debemos procurar tener una forma de brincar de un doc a otro para optimizar el almacenamiento de datos.

## Binary JSON

Aquí es donde aparece el binary JSON, un formato muy simple pero eficiente. La idea principal es conseguir que el documento sea más amigable y optimizado para el almacenamiento y la lectura a bajo nivel.

A la hora de construir el documento, lo primero que se hace es estimar el tamaño que tendrá.

En BSON, siempre se reservan 4 bytes que representan el tamaño del registro.

Luego, el documento JSON se convierte en un registro binario con campos definidos de la siguiente manera:

1. Campo de tipo:

Representa el tipo de dato del field.

Por ejemplo, si el field **x** es de tipo entero (**int**), se puede representar como **0x10**.

2. Campo del nombre de la clave:

En este caso, la letra **x** en la tabla ASCII corresponde a **0x78**.

A este valor se le añade un terminador que siempre será **0x00**.

Esto constituye la representación del nombre del campo del name.

3. Campo del valor:

El valor de un entero de 32 bits (**int32**) se representa con 4 bytes.

Por ejemplo: **01 00 00 00**.

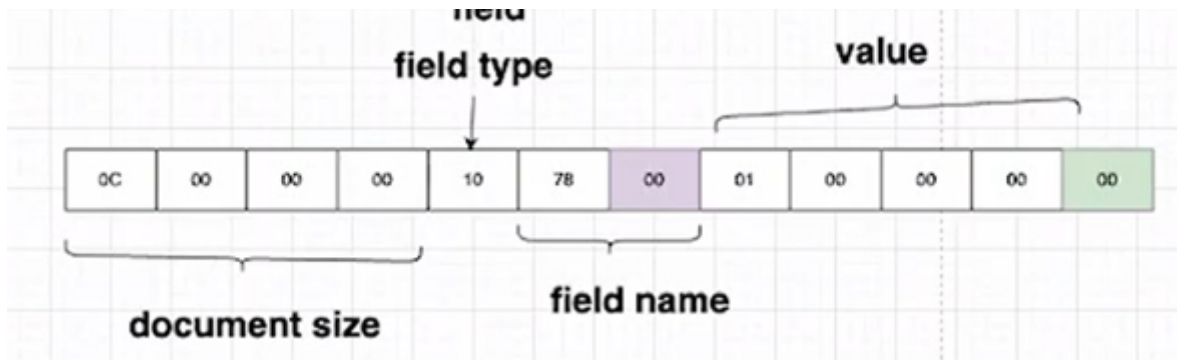
Esto se debe a que en disco se utiliza el formato Little Endian

4. Terminador del documento: Una vez que se termina de representar el valor, el documento se cierra con un terminador **0x00**.

Una vez que el documento está completamente representado, se cuentan los bytes totales para colocarlos en el campo de tamaño (**size**). Por ejemplo, un documento sería así:



Y el documento json codificado sería:



Donde el verde es el terminador de clave y el morado el terminador del doc. Y donde cada uno corresponde a un campo en específico.

Ahora, es importante mencionar que en Mongo puedo tener esquemas flexibles, es decir, los documentos no tienen que ser iguales entre sí. Cuando armo mi documento, los coloco uno detrás del otro. Imaginemos que tenemos varios documentos con diferentes tamaños. En este ejemplo, podrían ser 2315 bytes. Si estuviéramos en un esquema como el mencionado anteriormente, tendríamos que ir buscando hasta encontrar dónde empieza cada documento.

|      |  |
|------|--|
| 100  |  |
| 50   |  |
| 1500 |  |
| 65   |  |
| 100  |  |
| 500  |  |
| 2315 |  |

Ahora es diferente, porque en este tipo de documento siempre sabemos que el document size mide 4 bytes. Esto significa que siempre se leen esos primeros 4 bytes para saber el tamaño total del documento. Por ejemplo, si el tamaño dice que el documento mide 12 caracteres, el manejador de almacenamiento de Mongo lee esos primeros 12 caracteres y revisa si el documento contiene el campo que se está buscando. Si no lo encuentra, mueve el lector de disco exactamente 12 caracteres adelante, saltando al siguiente documento. Si ese tampoco es, repite el proceso saltando a la posición del siguiente. Este método hace que los saltos sean más eficientes, ya que no es necesario revisar byte por byte, sino que se avanza directamente al inicio del siguiente documento.

---

## Problema con los campos en un JSON

Pensemos en lo siguiente: todavía existe algo ineficiente en el almacenamiento, y es el nombre del campo. Por ejemplo, si tenemos un campo llamado **name**, ese nombre por sí solo ocupa alrededor de 4 bytes. Si además tenemos otro campo llamado **description**, este ocuparía aproximadamente 11 bytes. Entre ambos ya suman unos 15 bytes solo en los nombres de los campos.

Si los documentos tienen esquemas muy grandes, el problema se multiplica, ya que habrá muchos nombres de claves repetidos. Esto se vuelve contraproducente porque, si tenemos un millón de documentos, serían aproximadamente 15 millones de bytes desperdiciados solo en nombres de campos que se repiten y no cambian. Por esta razón, un motor como MongoDB implementa mecanismos de compresión para lograr un almacenamiento más eficiente, reduciendo el espacio que ocupan estos nombres de campos repetitivos.

## Dato sobre datos binarios para búsquedas

Aunque los datos sean binarios, eso no significa que no se puedan usar para realizar búsquedas. Lo que sucede es que requieren otro tipo de tratamiento.

**Imagen** Por ejemplo, cuando tenemos una imagen, podemos pasarla por un OCR (Optical Character Recognition).

El OCR se encarga de reconocer el contenido de la imagen y extraer todo el texto que contiene.

Ese texto luego se puede incorporar dentro del documento JSON, y al realizar búsquedas sobre ese campo, se podrán obtener resultados relacionados con la información contenida en la imagen.

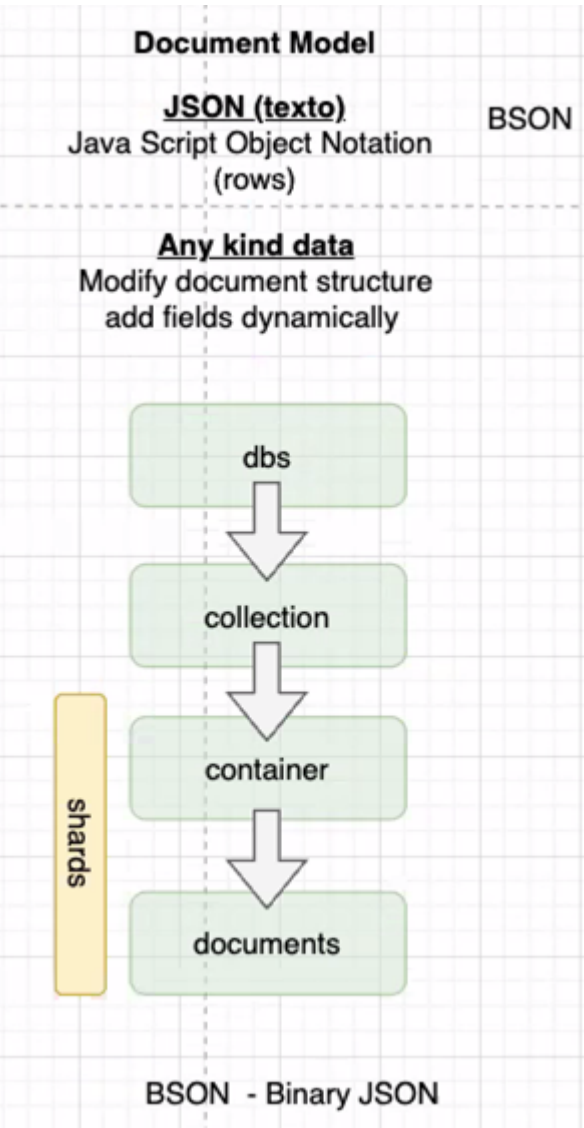
**Videos** Lo mismo ocurre con los videos, que están compuestos por un conjunto de frames o imágenes. Se pueden procesar mediante un OCR que extrae el texto e identificamos el tiempo donde el texto aparece y una vez que se recupera toda la info, se puede desplegar a la persona el video con el desplazamiento donde se encuentra el texto.

**Audios y comprimidos** En el caso del audio, se puede aplicar un proceso de speech to text para convertir la voz en texto y luego extraer los datos. Si se trata de archivos comprimidos como tar o zip, basta con extraerlos antes de procesarlos.

## Orden de una base de datos

Podemos decir que en MongoDB una base de datos se puede entender como una colección, que es que un contenedor que contiene documentos.

Cuando hablamos de un sistema relacional, la base de datos es una colección y el contenedor sería la tabla. En cambio, en MongoDB, la base de datos contiene colecciones, y cada colección es equivalente a una tabla. Dentro de esa colección, cada documento equivale a una fila/row en una base de datos relacional.



---

## Shards y Chunks

**Shard**

Cada shard guarda una parte diferente de los documentos. Esto ayuda a que el trabajo se reparta entre varios servidores y las consultas sean más rápidas.

**Chunk**

Mongo usa los chunks para decidir cómo repartir los datos entre los diferentes shards. Cuando un chunk se llena mucho, Mongo lo puede dividir en dos y mover uno de ellos a otro shard para mantener todo equilibrado.